

OptimStore: In-Storage Optimization of Large Scale DNNs with On-Die Processing

Junkyum Kim^{1,2}, Myeonggu Kang², Yunki Han², Yang-gon Kim^{2,3}, and Lee-sup Kim²

¹SAIT, Samsung Electronics

²School of Electrical Engineering, KAIST

³System LSI, Samsung Electronics

jkant215.kim@samsung.com, {mgkang95, yunki.han, leesup}@kaist.ac.kr, yanggon.kim@samsung.com

Abstract—Training deep neural network (DNN) models is a resource-intensive, iterative process. For this reason, nowadays, complex optimizers like Adam are widely adopted as it increases the speed and efficiency of training. These optimizers, however, employ additional variables and raise the memory demand $2\times$ to $3\times$ of model parameters, worsening the memory capacity bottleneck. Moreover, as the size of DNN models is projected to grow even further, it is not practical to assume that the future models will fit in accelerator memory. This has triggered various efforts to offload models to flash-based storage. However, when the model, especially the optimizer, is offloaded to flash, the limited I/O bandwidth severely slows down the overall training process. To this end, we present OptimStore, a solid-state drive (SSD) system with on-die processing (ODP) architectures for gradient descent-based machine learning models. OptimStore accelerates the training process of such large-scale models by processing model optimization in the storage device, specifically inside the flash dies. ODP capability of OptimStore eliminates the heavy data movement over external interconnect and internal flash channels. Overall, OptimStore achieves, on average, a $2.8\times$ speedup and a $3.6\times$ improved energy efficiency in the weight update stage over baseline SSD offloading.

I. INTRODUCTION

Since the advent of BERT [62], transformers have become the dominant choice of DNN model for various natural language processing (NLP) tasks, replacing the recurrent neural networks (RNNs). While transformers and RNNs are both designed to handle input data of sequential order, transformers can process input sequences in parallel, thanks to the attention mechanisms. Furthermore, transformers are free from the vanishing gradient problem as each layer has access to all the input tokens. Transformers allowed the NLP community to train larger models on a larger dataset, which was previously impossible with RNNs.

Early transformers [48], [62] already had sizable model parameters ranging from a few million to several hundred million but could still fit in the GPU memory alone. However, as it has become apparent that larger models bring superior performance on downstream tasks, the deeper-the-better trend accelerated the growth of model sizes up to billions of parameters, far exceeding the slowly improving GPU memory capacity. Figure 1 clearly shows this disparity between supply and demand of memory space. While model size grew more

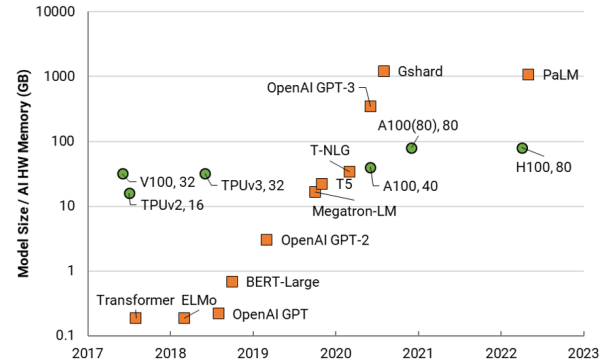


Fig. 1: Trend in DNN model size and AI accelerator memory

than $5000\times$ in recent five years, AI accelerator memory capacity scaled only by $2.5\times$.

This so-called memory capacity wall [32] has become the primary limiting factor in training large-scale NLP models, while optimizers, which are responsible for weight parameter update, are the most significant contributor to such issue. Recent DNN models employ advanced optimizers like Adam [27] to speed up the training process. These optimizers use additional variables to assist weight update computations. As a result, their memory footprint is several multiples of the model parameters themselves and accounts for a significant portion of the total memory requirement.

In contrast to its large memory demand, the weight update operation has low arithmetic intensity as all the computations are elementwise vector arithmetics. Therefore, using multiple GPUs to split and load optimizers in their memory would severely underutilize expensive GPUs during the optimization stage. Furthermore, although offloading approaches like [2], [15], [16], [47], [52], [53] can secure memory space by migrating tensors to the lower stage of the memory hierarchy, their focus is often limited only to buffering of intermediate tensors like activations and lacks consideration of the optimizers. Deepspeed [44], the open-sourced deep learning library, does support an optimizer offloading to storage. However, even the latest storage devices with PCIe 4.0 can support read bandwidth only up to 7.3GB/s [54], an order of magnitude slower than the DRAM. As a result, Deepspeed's offloading option suffers from impractically long storage I/O latency.

This work was done when J. Kim and Y. Kim was at KAIST

Many previous works [5], [24], [33], [36] have introduced in-storage acceleration for deep learning tasks by exploiting aggregated internal bandwidth of the flash channels. However, prior in-storage processing (ISP) architectures are limited in that these solutions are bottlenecked by data movement over flash channels when I/O workloads are highly sequential in the low-latency flash memories.

To this end, we propose OptimStore, a comprehensive storage system with on-die processing capabilities for the optimizer step of large-scale DNN training. In OptimStore, the weight update happens inside the flash die, mitigating the memory capacity requirement and alleviating the heavy data movement over the flash channels. OptimStore consists of flash memory chips capable of on-die processing, an iteration manager, a simple SSD firmware extension for managing optimizer steps, and a lightweight FTL for DNN training.

An OptimStore die has unified processing elements (PEs) for every plane and a shared bus provides access to distant planes. OptimStore die is carefully designed considering integration to Open NAND Flash Interface (ONFI) protocol. Its added components do not interfere with or affect existing functionality and can be controlled by the conventional commanding interface. We also present an FTL tailored for DNN optimization stages. This FTL performs L2P mapping with a much smaller mapping table and simplified wear-leveling and garbage collection with minimal effect on the performance of weight update operations.

To the best of our knowledge, OptimStore is the first on-die processing architecture to accelerate the optimization stage of large-scale DNN with practical design considerations. We summarize the contributions of our work in the following:

- We quantitatively profile the optimizer offloading onto storage when training large-scale DNN and identify the architectural bottleneck and possibilities of near-flash processing of weight update operations.
- We propose OptimStore, an on-die processing architecture for the optimizer step. OptimStore effectively utilizes the internal parallelism of SSD by isolating data traffic inside the flash die.
- We provide how OptimStore is integrated into the existing ONFI protocol, present a customized FTL for OptimStore, which operates in the optimizer step, and confirm that its overhead has minimal effect on the performance of DNN training.
- Using the state-of-the-art SSD simulator SimpleSSD [13], we model and evaluate OptimStore. OptimStore provides $2.8\times$ speedup and reduced energy consumption by $3.6\times$ in the optimizer step of large-scale transformer models compared to the CPU-SSD baseline and $1.9\times$ speedup over the ISP solution [41].

II. BACKGROUND

A. Training Process of DNN Models

DNNs are neural networks with multiple layers, and the depth of these networks can go as deep as hundreds of

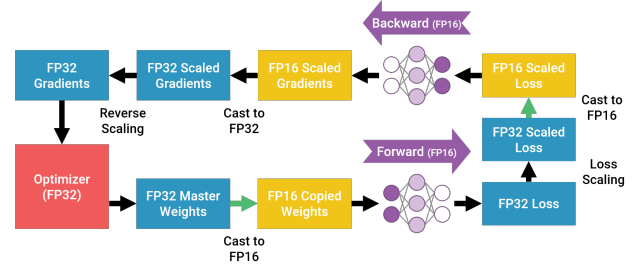


Fig. 2: Workflow of mixed-precision training

layers [3], [56], [59] which results in huge memory footprints. The DNN training process can be largely divided into three parts in the following order: Forward propagation, backward propagation, and the weight update stage. However, more steps are added when the DNN model is trained in mixed-precision [43] and with advanced optimizers.

Mixed-precision training is a widely applied technique to speedup GPU math operations for large-scale DNN models [8], [19], [30]. Figure 2 illustrates the procedure of mixed-precision training. First, master weight, the FP32 model parameter, is copied and cast into half-precision weight. GPU performs forward propagation in half-precision with these weight copy and obtains a FP32 loss value at the end of the forward pass. Before quantization, the loss value is scaled to prevent the underflow of gradient during the backward pass, due to the reduced representation window of half-precision. GPU then begins backward propagation and computes scaled FP16 gradient. Since the optimizer step is performed in FP32 for better accuracy, the gradient is de-quantized and unscaled to their proper value. Finally, the optimizer performs weight update operations with the FP32 gradient, master model weight, and possibly other optimizer state variables.

Earlier models employed stochastic gradient descent (SGD) as optimization method, which simply subtracted gradients from the weight parameters. However, nowadays, it is common to use advanced optimizers to enhance training accuracy and speed up convergence of parameter values. These optimizers are usually composed of multiple state variables, including the master model weights. The most widely used optimizers are the Adam [27], Adagrad [11], and RMSProp [9]. Unfortunately, these optimizers are memory capacity hungry and bandwidth-hungry [46], [51].

Although the update algorithm of the Adam optimizer is a combination of simple elementwise addition and multiplication, the FP32 optimizer states and gradient take up to eight times more capacity than the FP16 inference weight used in the forward propagation stage. Furthermore, as model parameters and gradient must be saved in both precision right before the optimizer step, the peak memory demand increases by another magnitude when mixed-precision is combined with an advanced optimizer.

B. Modern SSD Architecture

The organization of modern SSD is illustrated in Figure 3 (a). SSD has a hierarchical memory structure to increase I/O

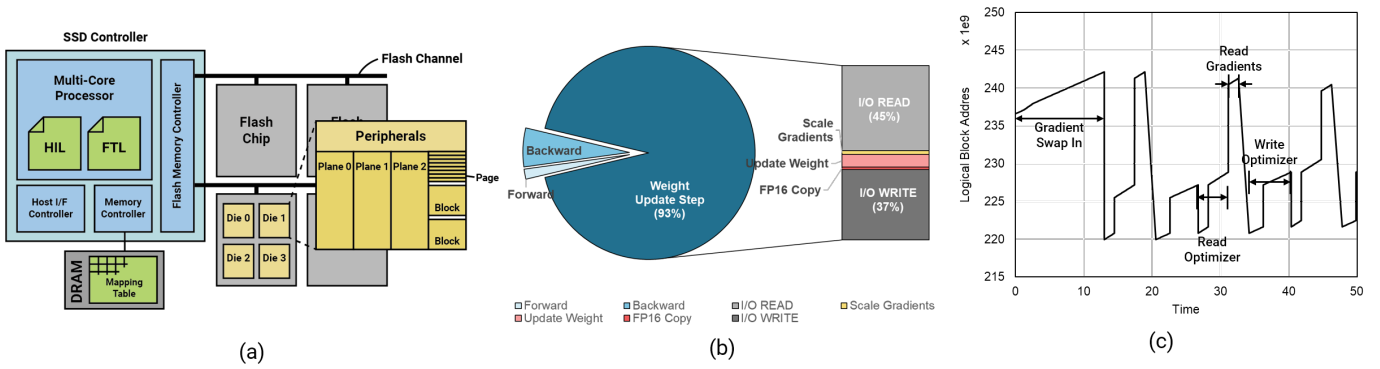


Fig. 3: (a) Architecture of modern SSDs (b) latency breakdown and (c) accessed logical block address during training GPT2

performance. A flash chip consists of several dies connected to the channel via a multiplexing interface. These dies communicate with the controller over the flash channel. Since the channel is a simple byte-wide bus, only one die can communicate with the controller at a time. Still, they can benefit from structural parallelism by receiving commands sequentially and processing the requests concurrently.

Each die has 2 to 16 planes. A plane is a memory array whose width is equal to page (4KB~32KB), the basic unit of flash operation. A pair of registers are present in each plane to buffer data read from or to be written to the flash array. Since these planes share the word line decoder, pages on different word lines cannot be accessed simultaneously. However, if pages on the same word line are accessed, multiple planes can be activated simultaneously to maximize parallelism.

The controller of SSD is an embedded multicore processor that runs internal firmware called flash translation layer (FTL). FTL bridges the semantic gap between host I/O requests and SSD's internal command. FTL translates the address of the request from the logical memory space to the actual physical flash page by referring to the mapping table. FTL also performs flash memory management tasks like garbage collection to free up invalid flash blocks and wear leveling to prolong the service lifespan of the device.

Recent SSDs employ triple-level cell (TLC) or quad-level cell (QLC) NAND flash dies to provide larger storage space. In order to compensate extensive access latency of TLC/QLC flash dies, many of them have a pseudo-SLC (pSLC) cache area. pSLC uses TLC or QLC NAND flash cells but stores only a single bit of data in each cell, trading off capacity with shorter access latency, close to the normal SLC flash's $tR(\approx 30\mu s)$ and $tPROG(\approx 100\mu s)$ [57].

C. In-Storage Processing

Instead of moving data to compute, near-data processing moves computations to data, as its name suggests. Simple computations like addition and multiplication can be easily conducted by a simple low-end core. In-storage processing (ISP) [14], [24], [33], [35], [36], [40], [41], [64] is a branch of NDP, where computations are performed inside the storage devices. Computation near non-volatile storage is the focus

of study in this paper, but we differentiate OptimStore from conventional ISP [24], [33], [35], [36], [41], [64], as they perform computations with embedded cores or additional logic placed near the controller, and not inside the flash chips. The key idea of ISP is to harness the power of aggregated internal bandwidth and reduce data movement over slow external interconnect. However this benefit is limited by the number of channels, hence ISP often fail to fully utilize structural parallelism of SSDs.

III. MOTIVATIONS

In this section, we describe our study's motivations and a simple evaluation of the existing optimizer offloading technique to show its capability and limitations.

In-applicability of CPU Memory The straightforward solution to the GPU out-of-memory error is offloading tensors to relatively spacious system memory. However, in many cases, CPU memory is not sufficient to support large-scale DNN training. For instance, the baseline system we evaluated has CPU memory of 128GB and 24GB of GPU memory space. Nevertheless, it is impossible to train an 8B parameter model with the Adam optimizer in this system. It requires 119GB of memory to store data for the optimizer step, and FP16 parameters and gradients add 29.8GB more, resulting in peak memory demand that exceeds the entire system memory pool.

Providing additional DRAM to the CPU can solve the problem. However, unfortunately, DRAM scaling has not been keeping up with the exponential growth in the demand for more capacity. Thus, we look ahead and study scaling and training large DNN models inside flash to target the upcoming problem given such recent trends.

On top of that, equipping the CPU with TB-scale DRAM calls for measures to utilize such memory. However, prior work [2] has demonstrated that execution of other workloads in parallel leads to performance degradation for CPUs. This means CPUs that have been given large memory either need to face performance degradation or resource underutilization, and both are not appreciated from the server's perspective, justifying our argument for offloading the weight update stage in training to the flash side.

SSD Offloading Profiling Results We profile the I/O pattern when offloading the optimizers to the SSD using Deepspeed's

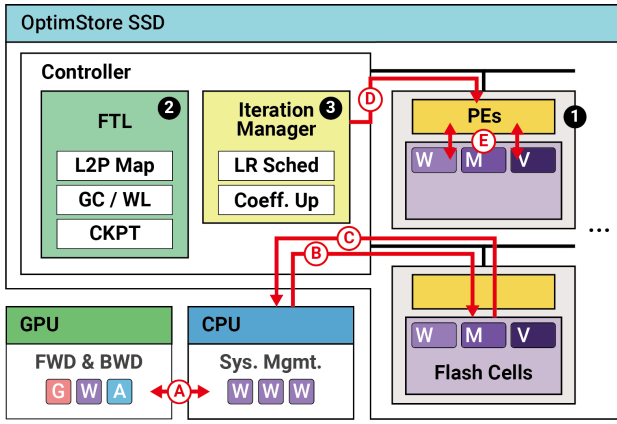


Fig. 4: Overview of OptimStore SSD and its components

[44] SSD offloading option. As CPU memory space is not enough to hold both weight parameters and the optimizer states, we prioritize half-precision model parameters and allocate them into more accessible CPU memory. Optimizers are offloaded to the SSD. Figure 3(b) shows the runtime latency breakdown of an iteration during fine-tuning of GPT2 [49] with 6.7B parameters. As the weight update step takes up to 93% of the total process, accelerating the weight update step can drastically improve the overall training time performance. Moreover, the I/O latency accounts for 82% of the overall weight update step execution time. Thus, we conclude that alleviating the data read and write bottleneck will lead to a significant performance boost in the overall training system when using the SSD offloading option.

The overall I/O access pattern is plotted in Figure 3(c). A sequential read and write of constant size dictated the overall pattern, with occasional and rare jumps, indicating access to different optimizer state variables or gradients. This implies that despite the I/O step severely holding back the overall execution speed, its access pattern is predictable and straightforward. In addition, as the tensors are significantly large (the smallest tensor of 6.7B GPT model is 64MB) slicing and processing tensors in page-sized batch would incur minimal overhead. Furthermore, as weight update operations are elementwise computations, data within the page is sequentially accessed. Hence byte addressability or fine granule data access is unnecessary for optimizer step inside the flash memory.

Bus Contention in Flash Channels The flash interface has a relatively slow transfer speed of around 800MT/s to 1,200MT/s [17] and narrow channel bus width of a single byte or a word. In many cases, this does not limit the SSD performance as I/O requests are random and small-sized, so the main bottleneck of flash operation is usually the latency of the flash array. However, if the I/O workload is highly sequential and most flash accesses are performed in multi-plane mode operations, data movement over the flash channel becomes the primary factor that hinders the performance. When writing or reading super-pages in the pSLC cache area, this becomes more severe.

We obtained an average latency breakdown of internal

request of flash memory using SimpleSSD. The same configuration is used as in section VI. For read operations, 19.52% of the time request latency was caused by delays from bus contention of the flash channel before reading the data. After reading the page from the flash, 9.67% of total request latency was spent waiting for the channel to be empty. In the case of write operations situation was worse, as the flash die was left idle for 66.05% of the total request time waiting for the channel to be available. As a result, if the I/O pattern is deterministically large and sequential, reducing data transfer over the channel is crucial in order to utilize SSD's structural parallelism maximally. To minimize storage-host data transfer and DMA latency over the flash channels, we propose the idea of applying NDP at the flash die level as an attempt to ease the overall system bottleneck.

IV. OPTIMSTORE ARCHITECTURE

A. Overview

The overall architecture and workflow of OptimStore SSD are shown in Figure 4. OptimStore constitutes a heterogeneous computing system for DNN training along with CPU and GPU. OptimStore is responsible only for the optimization stage, while GPU performs forward and backward propagation. CPU manages the entire execution process and the tensor migration from and to OptimStore SSD.

The main feature of OptimStore SSD is the NAND flash dies ① with PEs capable of performing floating-point operations. A customized FTL ② manages logical to physical mapping of optimizer state variables and the metadata for model checkpointing. Other than the FTL, the controller runs the iteration manager ③, which is a simple firmware that includes a learning rate scheduler and the mixed-precision gradient scaler. It manages constants required for the weight update operation, such as learning rate and bias correction.

The overall process of DNN training using OptimStore SSD is as follows. In the backward propagation, the GPU computes FP16 gradients and `memcpy` (A) them to the CPU. The CPU then casts gradient to FP32 gradient and swaps them out to the flash memory (B). As the gradients are swapped into the flash dies, the on-die PEs perform weight update computations and write them back into flash (E). Finally, after all the weight parameters of the model are processed, OptimStore begins swapping out the new weights to the CPU (C). In the following forward pass, the iteration manager updates the constants and broadcasts them to flash chips (D).

B. On-Die Processing Architecture

Figure 5 (a) shows the functional diagram of the four-plane OptimStore die. Blocks colored in grey indicate logic components added in the OptimStore. PEs are placed in a per-plane manner. With dedicated PE for each plane, OptimStore can exploit plane-level parallelism when performing computations. The PE and the cache register in the same plane are connected by a local bus separate from the plane's I/O path. Conventional flash dies do not support data transfers between different planes. However, in OptimStore, the input ports of

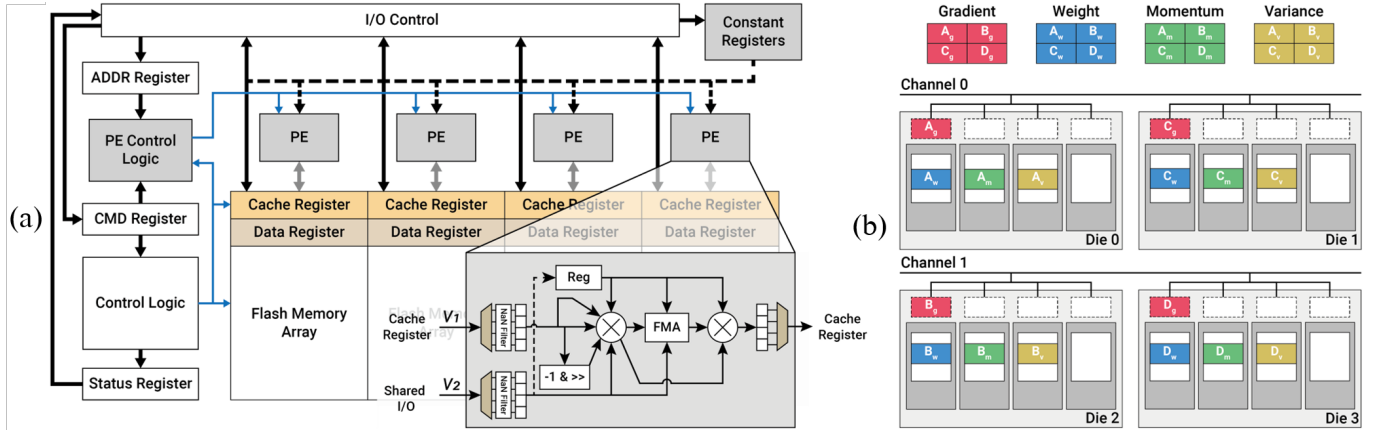


Fig. 5: (a) Functional block diagram OptimStore die and (b) data mapping of Adam optimizer in OptimStore

PEs and the I/O path of each plane are connected by a shared bus, giving PEs access to remote planes. PE control logic can select the source cache registers and destination PE by turning on and off the switches of the shared bus.

The microarchitecture of PE is also depicted in Figure 5 (a). A subtract-shifter, two multipliers, and a fused multiply adder constitute the pipe-lined PE. With these components, it can perform all the necessary computations for the weight update operation in the efficient forms of fused operations, such as linear vector sum (LVS) and quadratic vector sum (QVS). The inverse square root is implemented with the fast inverse-square root algorithm [63]. In addition, PEs have internal registers to store constants needed for computations throughout an iteration of the optimizer step.

In NAND flash dies, data is transferred byte by byte or word by word. On the other hand, OptimStore's PEs are designed to handle 32-bit floating-point weight update operations. Thus it takes four data transfer cycles for a single input. For this reason, PEs operate at one-fourth of the NAND flash interface frequency. Limited by the data transfer rate, PEs do not require vector processing capability, and hence scalar units are sufficient. Hence OptimStore is implemented with minimal overhead; 0.15% of the original flash die area. Moreover, as on-die ECC NAND flash with per-plane ECC logic [42], is a commercialized design similar to ours, we claim that OptimStore's on-die PEs are also an implementable and feasible architecture.

Fused Optimizers Executing a single on-die computation command consumes many cycles due to the slow byte-by-byte data transfer rate of the NAND flash interface. Therefore it is crucial to optimize the computation data flow and reduce the number of PE commands as much as possible. Algorithm 1 elaborates the fused Adam optimizer tailored for on-die computations. In the original form, Adam first updates the momentum and variance values and saves them as part of the optimizer state variables. However, these values require bias correction before being used for weight update. We combine momentum and variance update with bias correction into a single operation to avoid tedious floating-point divisions inside the OptimStore dies. Moreover, rather than saving raw momen-

Algorithm 1 Fused Adam Optimizer

Input: $\theta, g, \hat{m}_\gamma, \hat{v}, \beta_1, \beta_2, \gamma, iter, scale$

Output: $\theta, \hat{m}_\gamma, \hat{v}$

Parameter: $page_offset, page_size$

```

1:  $a_m \leftarrow \frac{\beta_1(1-\beta_1^{iter-1})}{1-\beta_1^{iter}}$   $b_m \leftarrow \frac{1-\beta_1}{1-\beta_1^{iter}} \times scale$ 
2:  $a_v \leftarrow \frac{\beta_2(1-\beta_2^{iter-1})}{1-\beta_2^{iter}}$   $b_v \leftarrow \frac{1-\beta_2}{1-\beta_2^{iter}} \times scale^2$ 
3: if  $\gamma \neq \gamma_{prev}$  then
4:    $a_m \leftarrow a_m \frac{\gamma}{\gamma_{prev}}$   $b_m \leftarrow b_m \frac{\gamma}{\gamma_{prev}}$ 
5: end if
6: for i in  $page\_offset$  to  $page\_size$  do
7:    $\hat{m}_\gamma[i] \leftarrow LVS(\hat{m}_\gamma[i], g[i], a_m, b_m)$ 
8:    $\hat{v}[i] \leftarrow QVS(\hat{v}[i], g[i], a_v, b_v)$ 
9:    $\theta[i] \leftarrow \theta[i] - ISQRT(\hat{v}[i]) \times \hat{m}_\gamma[i]$ 
10: end for

```

tum and variance values, OptimStore Adam optimizer stores bias-corrected values of momentum and variance to remove writing in between the computations. The fused optimizers are further combined with learning rate scheduler (Line 3~5) and gradient scaling (Line 1~2).

C. Data Layout

We design OptimStore based on flash dies with four or more flash planes, typical of high-performance SSDs. With such configuration, OptimStore can support the Adam optimizer that has three types of state variables. Furthermore, to exploit flash-level parallelism, we adopt data mapping similar to the multistream SSDs [6], placing different state variables in different planes.

The size of weight tensors in large-scale DNN models is at least several tens of MB. Thus, as mentioned in section III parameter tensors are split into a number of pages when stored in the flash memory. These pages are first stripped across multiple channels and die. Each optimizer state tensor is allocated to a different plane of a NAND flash die. Figure 5 (b) shows an example data layout of a set of state tensors of

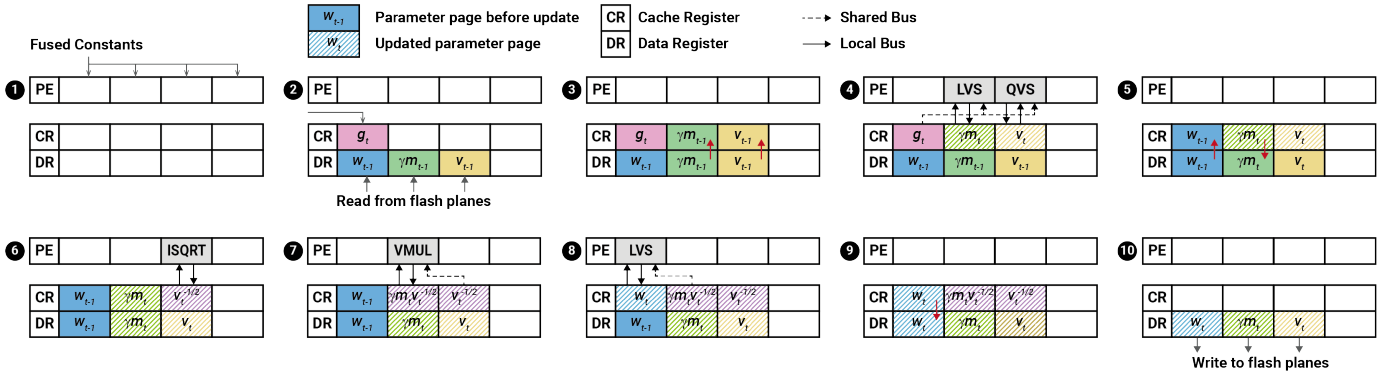


Fig. 6: Computation flow of DNN optimization stage in OptimStore SSD

Adam optimizer. Page from momentum and variance tensors are located on the same word line offset of corresponding master weight and grouped as a single super-page inside the flash die. Meanwhile, gradient, which has a temporary use, is only written on the cache register of a flash plane where the master weight resides. By having all the required pages for weight update operation in a super-page, OptimStore die can read operand pages and write back the result, each with a single multi-plane operation.

Tensors may have sizes that are not divisible by page. However, this does not affect the performance of OptimStore, since a group of optimizer state variables also shares the same byte offset within the page. The PE can process them agnostic to the layer and tensor boundaries. If the number of state variables that constitute the optimizer is different, data is placed slightly differently. For instance, the RMSProp optimizer only has the master weight and the variance as state variables. Thus, two of the four planes are allocated for momentum, and the other two stores the weight. In this case, two pages of gradients are swapped in and consumed at each instance of the weight update operation.

D. Optimization Procedure

Figure 6 describes the training process on a OptimStore with an example for clear understanding. At the beginning of each iteration, the iteration manager recomputes the fused coefficients ①. These values are then broadcasted to all dies in the OptimStore SSD are stored in each die's constant register.

After the forward pass, the host CPU sends the flattened gradient tensors to OptimStore SSD as a form of I/O request. This request is split into internal sub-requests in page granularity by internal firmware running on the SSD controller. For each sub-request, gradients are written into the cache register on the master weight plane. Then the corresponding optimizer state variables are read from the flash to data registers ②.

On-die computation starts with copying momentum and variance pages to the cache registers for computation ③. Page movement between the two registers typically takes around $3\mu s$ (tPCBSY) [17]. PEs then concurrently calculate bias-corrected new momentum and variance values using their previous values and the gradient value, which are broadcasted from the first cache register via the shared bus ④. Next, while

updated variance and momentum values are dumped into the data registers, weight parameters are copied into the cache register ⑤. Gradients are cleared from the cache register at this time of operation. Following the cache register copy, PEs perform a sequence of commands - inverse square root ⑥, vector multiplication ⑦, and vector addition ⑧. The resulting updated master weight is then dumped into its data register ⑨. Vector addition is not defined in Table I, but it is implemented by supplying ones as coefficients in LVS operation. Finally, the updated optimizer states residing in the data register are written back to the flash memory array ⑩.

V. SYSTEM SUPPORT FOR OPTIMSTORE

A. OptimStore ONFI Commands

Accessing NAND flash memory takes a long time, ranging from a few microseconds to hundreds of microseconds. OptimStore mitigates this issue by actively utilizing internal registers of NAND flash dies. We propose a new set of NAND flash commands to support OptimStore operations as an extension to the existing ONFI protocol.

Volatile Page Program (VPP) operation provides a faster and flash-friendly way to temporarily store short-lived data such as the gradients. These data are stored only in the cache register rather than written to the flash memory. Register clear signal is inactivated until the next non-OptimStore command is issued.

Buffered Page Read/Program (BPR/BPP) commands are similar to copyback read, and copyback write operations but different in that BPR and BPP store page only in the data registers and do not copy them into the cache registers.

On-Die Computation (ODC) When on-die computation commands are issued, flash arrays remain idle along with the die control logic, and PE control takes over the control path. On-die computation operations require a single command DQ cycle for an opcode and four address DQ cycles. The two-column address cycles identify the byte offset within the page as before. The first row address cycle specifies two operand planes, while the last row address cycle is used to select a constant value from the constant registers. We list OptimStore NAND flash commands including on-die computations in Table I. The opcodes are chosen from a list of vendor specific commands referring to ONFI 5.0 specification [17].

	Command	Opcode	Description
ODC	LVS (v_1, v_2, α, β)	01h	$v_1 = \alpha v_1 + \beta v_2^2$
	QVS (v_1, v_2, α, β)	02h	$v_1 = \alpha v_1 + \beta v_2$
	VMUL ($v_1, v_2, \cdot$)	03h	$v_1 = v_1 \odot v_2$
	ISQRT ($v_1, \cdot$)	04h	$v_1 = 1/\sqrt{v_1}$
	PCACHE	16h	Dump page: CR $\rightarrow$ DR
	RCACHE	17h	Copy page: CR $\leftarrow$ DR
	VPP	13h	Volatile Page Program
	BPR	19h	Buffered Page Read
	BPP	1Ah	Buffered Page Program

TABLE I: ONFI commands for OptimStore operations. CR and DR denotes cache register and data register respectively

B. OptimStore FTL

During the DNN training, a portion of SSD sectors are reserved only for storing the optimizers. This space serves as a short-term memory reservoir for computations, where tensors are stored only for a few iterations. OptimStore’s FTL employs a separate mapping table that manages blocks in the reservoir.

The mapping table for optimizers is smaller since the pages are mapped in super-page and super-block granularity. Optimizer tensors are saved as a grouped data structure, where pages of corresponding tensors constitute a 4-plane wide super-page. Since all dies share the same data layout, a universal, shared *plane number* is enough to address different optimizer variables within the flash die. For this reason, the mapping table for the optimizer is reduced by a quarter of the size of the original mapping table.

Garbage Collection OptimStore undergoes a heavy load of the program and erases during the DNN training. Nevertheless, as all the programming is performed sequentially, garbage collection (GC) overhead in OptimStore is minimal, with a write amplification factor close to one. Garbage collection is triggered when the newly updated optimizer tensors are swapped out from the SSD to secure storage space for future weight update computations. Although the latency of erase operation is significantly larger than other flash operations when GC is triggered, OptimStore SSD performs erasing operations at multiple super-blocks in every dies simultaneously. Therefore the overall latency of GC is shorter than the forward pass and completely overlaps with GPU computations.

Wear Leveling OptimStore FTL conducts two types of wear leveling (WL), inter-die WL and intra-die WL. Although optimizer tensors are distributed almost evenly across the dies, several dies may end up storing a few more pages of tensors than others. As DNN training requires hundreds of repeated iterations, this difference is amplified and adversely affects the service life of the underlying SSD. Inter-die WL mitigates such issues by migrating super-pages between the dies. It requires a few super-page read and writes and can be hidden in the forward pass.

Unlike inter-die WL, the necessity of intra-die WL depends on the optimizer type. For optimizers like Adagrad and RM-SProp, all four planes are utilized in the optimization stage and thus do not require extra load balancing. In contrast, the Adam optimizer only utilizes three planes at a time, leaving one plane idle during the optimizer step. For this reason, when

Adam optimizer is in use, FTL rarely but regularly triggers intra-die WL. This is achieved simply by writing computation results to previously idle flash plane via shared bus, which consumes extra cycles but is still cheaper than conventional page copying methods.

Model Checkpointing As the model parameter grew, the cost of model checkpointing has also become a burden since it requires writing hundreds of GBs of data to the storage in the middle of training. In OptimStore, model checkpointing is implemented in a much cheaper way, as the optimizer tensors are already stored in the persistent flash memory from the beginning. Flash blocks in short-term memory reservoir may suffer from a shorter cell retention time since it undergoes repeated, heavy programming and erasing. Therefore, for safer storage of model weights, OptimStore supports checkpointing by regularly writing the result from the PEs into flash blocks outside of the short-term memory reservoir.

C. Programming Support

OptimStore programming interface is implemented with existing `ioctl()` system call. Sequence of flash commands and required set of parameters for optimizer update step is stored as vendor specific features. When programmer modifies optimizer, the compiler translates the optimizer operations into corresponding sequence of OptimStore commands set as a NVMe feature data. In the initialization step of training, `ioctl()` is invoked via python module `fnctl`. Then the underlying device driver invokes `nvme_set_feature` command to modify OptimStore features. FTL refers to the `parameter_set` feature to reorganize mapping and management tasks, while flash memory controller generates flash command sequence according to the `command_sequence` feature, when on-die operation requests are given.

VI. METHODOLOGY

We show the benefits of OptimStore when integrated in the heterogeneous computing system for DNN training. We evaluate five systems including the OptimStore for comparisons. Real machines are used to measure forward and backward compute time, while the performance of storage devices is obtained from simulations.

- **Baseline(BL):** We chose Microsoft’s Deepspeed [44] library with NVMe offloading enabled as the baseline system. It writes gradient tensors into the NVMe SSD during the backward pass. In the optimizer step, the CPU reads a portion of optimizer state variables and gradients, computes new weights, and writes updated optimizer states back to the SSD. As CPU memory space is not sufficient to store the entire optimizer, the CPU repeats this process until all the parameters are updated.
- **Conventional ISP(C-ISP):** With a few exceptions [14], [40], in-storage processing systems [24], [33], [35], [36], [41], [64] perform computations on the embedded core or additional processing units attached to the controller. Either way, they use internal DRAM as working memory space. Among various conventional ISP architectures,

SSD Configuration	
Controller & Host I/F	3 ARM Cortex-A9 Cores NVMe, PCIe 4.0 $\times$ 4
DRAM	8Gb $\times$ 16 DDR4-1600, 4GB
Flash Memory	8 Channels, 4 Packages, 2 Dies, 4 Planes 682 Blocks, 1024 Pages, Page size = 8KB NV-DDR3 1200MT/s, 8bit Channel Bus
Timing Parameters (pSLC)	$t_R = 25\mu s$, $t_{PROG} = 120\mu s$, $t_{ERASE} = 1ms$ t_{PCBSY} , $t_{RCBSY} = 3\mu s$ Storage stack latency = $5\mu s$
Energy Parameters	Operation Voltage = 3.3V I_{READ} , I_{PROG} , $I_{ERASE} = 25mA$ $I_{BUSIDLE} = 5mA$, $I_{STDBY} = 10\mu A$ OptimStore PE = 4.66mW

TABLE II: System configurations

we implement GenStore [41] as a representative system since its design and data layout is suitable for a large sequential I/O workload. More specifically, we implement the double-buffering of GenStore using internal DRAM to overlap data fetching and computations.

- **DeepStore(DS)**: DeepStore’s [40] on-chip accelerator places the systolic array and scratchpad onto the read path of each flash die. The accelerator does not perform or support write operations as it is built for intelligent queries. For fairness, we reduced 4×32 systolic array into a column of 4 PEs only for elementwise operations.
- **OptimStore(OS)**: We implement OptimStore as mentioned in section IV. OptimStore SSD performs weight update operations inside its flash dies using gradients swapped out from the host. After all the update operation is done, new model weights are read to the host.
- **CPU Offloading (CPU)**: In addition to offloading-to-storage, we evaluate the performance of the CPU offloading system by setting the offloading destination of both parameters and optimizer states to the CPU memory.

Performance Modeling We model OptimStore and conventional ISP using the SimpleSSD simulator [13], a state-of-the-art simulator for SSDs. To implement OptimStore’s on-die processing capabilities we modify the simulator’s physical abstraction layer so that there exists a computing unit inside each flash die. Other internal layers that manages NVMe request parsing and internal DRAM were also modified in order to model end-to-end latency of storage systems. C-ISP is provided with additional core dedicated only for weight update operations. We modify the I/O block trace file obtained from the NVMe offloading training, add OptimStore-specific requests, and feed it to the simulator. Table II summarizes SSD configurations used in the experiment.

Real Machine Evaluations We conservatively measure latency and power from real machines for forward and backward pass of all systems and CPU weight update time for the baseline and CPU-offloading. We run the training on a system with Intel XEON Silver 4210R CPU with 128GB DDR4 DRAM and NVIDIA RTX3090 GPU with 24GB of memory.

Area and Power We implement OptimStore PE and its control logic in Verilog HDL. The design is then synthesized using the Synopsys Design Compiler [58] with Samsung 65nm process technology node to get an estimation of area and power

overhead of OptimStore dies. The area overhead of OptimStore logic units are negligibly small as it takes up only 0.15% of original NAND flash die [21]. We use power estimation model of internal components of SSD based on Mcpat [34] and Drampower [22], which is incorporated in the SimpleSSD. Energy parameters are also arranged in Table II.

Test Benchmarks We evaluate the benefit of OptimStore when training representative large-scale transformer models GPT2 [49] and BERT [62] with varying number of weight parameters ranging from 2.6B to 10.3B. Additionally we evaluate two T5 [50] models; 2.88B and 11.3B. We conduct mixed-precision training of language modeling with GPT2 and BERT using wikitext dataset and translation training task with T5 and wmt’14 NMT de-en dataset.

VII. EVALUATIONS

A. Performance

The execution time of a training iteration in the baseline system, conventional ISP, DeepStore, OptimStore, and CPU-offloading are depicted in Figure 7, normalized to that of OptimStore’s results. The solid bars in the graph represent the time consumed by the optimizer step, including the I/O latency. The patterned bar denotes forward and backward propagation’s execution time until the gradient swap out begins. Evaluation results of CPU-offloading of models with more than 6B parameters are not shown in the graph since it is impossible to run the training task due to insufficient system memory space.

We make three observations from the performance evaluation. First, regardless of the model size and type, the conventional ISP, DeepStore and OptimStore outperformed the baseline in the optimizer step at a nearly constant rate. C-ISP accelerated the weight update stage by $1.5 \times$ on average over the baseline. OptimStore provides $1.9 \times$ larger speedup than C-ISP, reaching $2.8 \times$ the baseline. On the contrary, on average, the speedup from DeepStore is $1.1 \times$, failing to achieve larger performance gain even with on-die computation capabilities. This is due to a lack of write support for on-die compute logic. As it can only support regular writes, gradient write skipping is impossible. Moreover, to write back computation results, it has to first read out the pages to the controller and then write to flash chips, which incurs significant data movement and conflicts along the flash channels. The benefit of C-ISP and OptimStore in the weight update stage comes from (1) reducing data movement along the external PCIe interconnect, utilizing aggregated internal bandwidth of flash channel and flash dies, and (2) skipping writing of gradient values.

Second, for models smaller than 6B parameters, CPU-offloading achieved the best performance for the optimizer step. However, when forward and backward propagation is taken into account, OptimStore performance surpasses the CPU-offloading. During the backward phase, gradient swap between the GPU and CPU takes place. Hence the CPU and its memory are busy handling the swap stream and limited to performing computation. For this reason, in CPU-offloading, the entire process of forward, backward, and weight update is

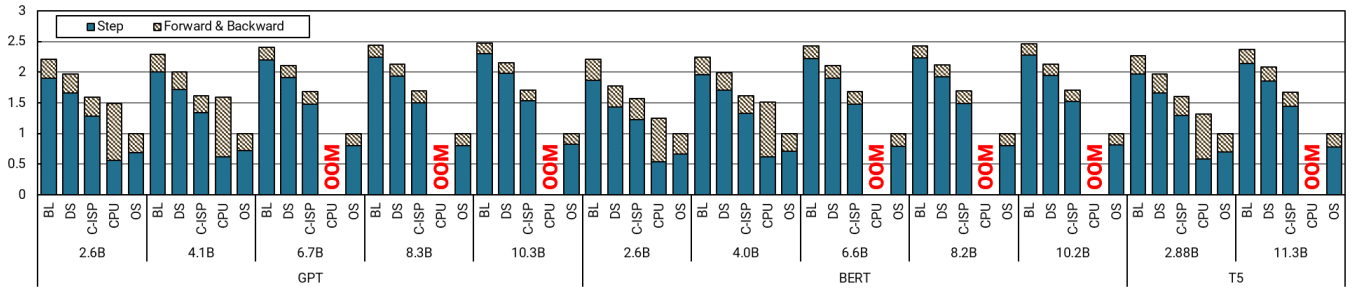


Fig. 7: Normalized execution time of the baseline, DeepStore, conventional ISP, CPU-offloading, and OptimStore

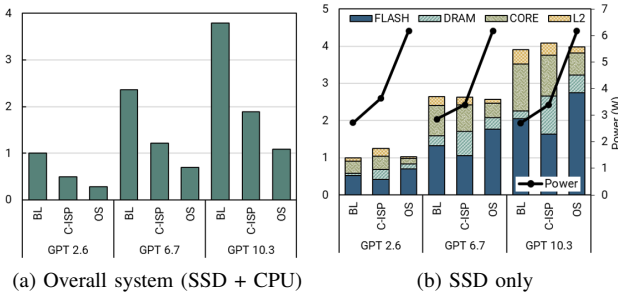


Fig. 8: Energy consumption during weight update

serialized, whereas, in OptimStore, the latter two stages are overlapped, resulting in a shorter overall execution time.

Third, when training larger models, the portion of the optimizer step dominates the end-to-end latency. Thus governed by Amdahl's law, the speedup of the entire training time by OptimStore and conventional ISP increases with the model size, from $2.2\times$, $1.4\times$ in the smallest model to $2.5\times$, $1.5\times$ in the largest model, respectively. Factoring in the forward and backward passes along with the optimizer step, OptimStore speeds up end-to-end training by $2.4\times$ on average.

B. Energy Efficiency

The energy consumption of three systems, the baseline, conventional ISP, and OptimStore, for the optimizer step, is shown in Figure 8. The Figure 8 (a) illustrates the energy consumed by the system involved in the optimizer step, including the CPU and the SSD. Since the power consumption of the CPU contributes most of the system power usage, OptimStore and C-ISP benefit from a shorter execution time, resulting in $3.6\times$ and $2.0\times$ reduced energy consumption, respectively. Extra energy savings larger than the speedup comes from the lower CPU utilization in C-ISP and OptimStore, as the CPU does not execute any intensive computations.

Figure 8 (b) depicts power consumption and the breakdown of the energy consumption of the SSD by its components - the flash, internal DRAM, L2 cache, and the embedded cores. The three systems resulted in similar energy usage, although conventional ISP and OptimStore have much shorter running times. The average power consumption of OptimStore is $2.2\times$ and $1.8\times$ larger than the baseline SSD and C-ISP, respectively. Compared to the baseline, C-ISP consumes less

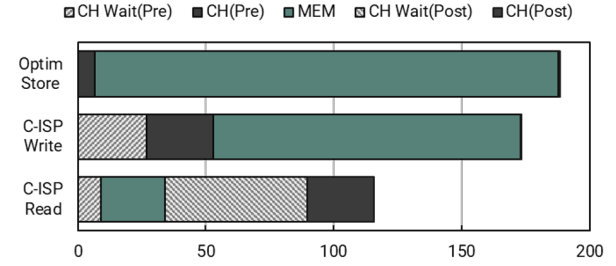


Fig. 9: Internal latency breakdown of flash operations

energy from flash memories due to shortened execution time and reduced number of flash accesses by skipping gradient writes. However, as it actively utilizes internal DRAM and embedded core for computations, C-ISP shows comparable or larger energy use from the cores, L2 cache, and the DRAM. In OptimStore the energy consumed by flash is larger than in the other two systems, as all the computations are performed in the flash dies. Still, the overall power consumption of OptimStore is well below the 75W PCIe power supply.

The data movement bottleneck can be clearly seen from the latency breakdown of OptimStore and conventional ISP flash operations. In the figure 9, the latency of OptimStore includes all the necessary operations for computations in the optimization stage of weights equivalent to a single page, whereas C-ISP read and write illustrates the latency of only the read and write operation of a super-page, respectively. Note that C-ISP requires both write and read operations to complete weight update operations. The latency of OptimStore is dominated by memory operation time (MEM) as all the computations are conducted inside the die. On the contrary, C-ISP write and C-ISP read requires constant DMA (CH) latency, and a significant portion of overall latency comes from waiting time due to busy channels, especially in read operations.

C. Sensitivity Test

Scalability In order to study the scalability of OptimStore, we evaluate the performance of OptimStore with the varying number of flash resources in the SSD. For comparison, conventional ISP and the baseline system are evaluated together. The experimental result of scalability is illustrated in Figure 10 (a). A SSD configuration with 4, 8 and 16 channels are used for the experiment. The number of dies is increased from 32

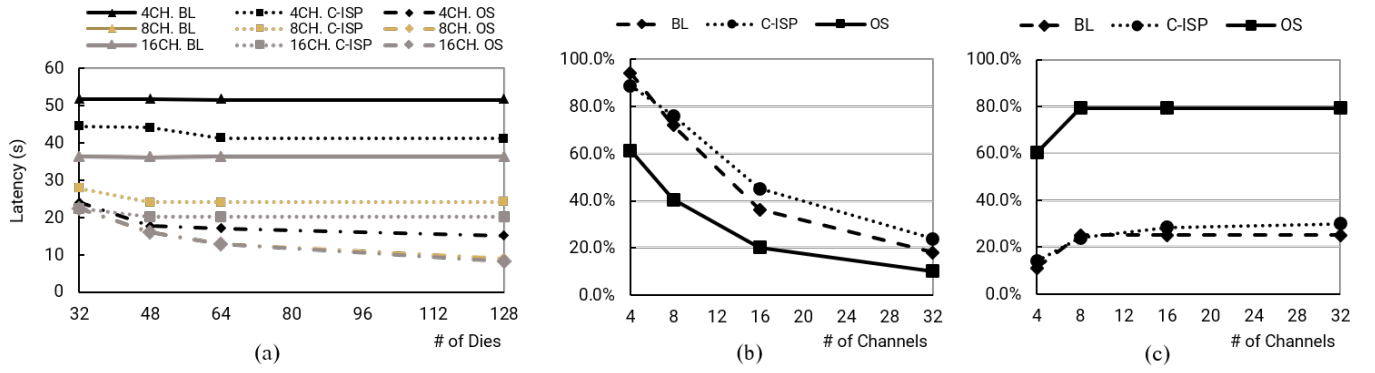


Fig. 10: (a) Scalability, change in (b) channel utilization and (c) die utilization with varying number of channels

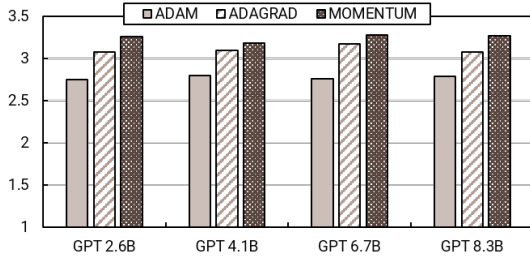


Fig. 11: Sensitivity test on various optimizer types

to 128 and the change in latency of the optimizer step was measured. BERT 6.6B model is used for evaluation.

Regardless of the number of dies, the baseline shows consistent performance. As the SSD reads and writes only in pSLC mode, the speed of flash operation is fast enough to saturate PCIe bandwidth even with 32 dies. Hence the baseline does not benefit from scaling flash resources. C-ISP's performance gain saturates starting from 64 dies on a 4 channel device, and 48 dies on 8 channels device while showing almost no improvement in 16 channel device. Doubled number of channels brings further speed up in the C-ISP, as it can utilize aggregated internal bandwidth of the SSD. However, its gain is limited as the write path cannot be effectively parallelized.

The poor scalability of conventional ISP becomes more apparent in Figure 10 (b) and (c), which shows the channel utilization and die utilization of the 64 die SSD, respectively, under a varying number of channels. With more channel buses, data traffic through each channel decreases, resulting in less bus contention. However, improvement in die utilization saturates around 30% in C-ISP even with lower channel utilization. This is because, although the increased number of channels reduces latency due to waiting for channel to be cleared, the latency of data transfer itself cannot be removed as computation takes place in the embedded core. In contrast to the baseline and the C-ISP, OptimStore shows a steady performance gain with the increasing number of dies, presenting a clear advantage in scalability.

Various Optimizers As OptimStore is designed to support other optimizers, we evaluate the performance of OptimStore when applied to optimizers other than Adam. We conduct

Model	Size	Intra-Die WL	Garbage Collection	
			Latency	Energy
GPT	6.7	13.5s (2.6%)	49ms	259mJ
	10.3	21.0s (2.6%)	76ms	401mJ
BERT	6.6	13.4s (2.6%)	49ms	259mJ
	10.2	20.8s (2.6%)	76ms	401mJ
T5	11.3	23.0s (2.6%)	84ms	444mJ

TABLE III: Result of FTL task overhead evaluation

experiments on four size GPT models, each with three types of optimizers - Adgrad and Momentum and Adam. The experimental results are summarized in Figure 11. Each bar graph shows speedup compared to the baseline. The Adam optimizer benefits the least from OptimStore as one of four planes is left idle during the optimizer step. On the other hand, Adagrad, Momentum can fully utilize the planes in the flash dies and their internal bandwidth, resulting in a larger speed up. Moreover, as the Momentum algorithm is implemented with fewer math operations, it has a shorter latency of on-die computations than the Adagrad. As a result, the Momentum benefits the most from OptimStore by $3.2\times$ over the baseline.

D. FTL Overhead Analysis

We evaluate the overhead of OptimStore FTL memory management tasks to ensure these supplementary features do not affect the overall performance of OptimStore. We focus on intra-die WL as the extra-step of migrating the page between the planes increases the time required for on-die computations by 18% in the optimizer step. We also verify that OptimStore's GC can be overlapped with the forward pass. We test intra-die WL and OptimStore GC with two GPT models, two BERT models, and an 11.3B T5 model. The results are shown in Table III. The intra-die WL incurred additional latency of only 2.6% of the original execution time. Latency of garbage collection is confirmed to be short enough that it can be completed within the forward pass.

VIII. DISCUSSIONS

SSD Lifetime & Endurance Previous studies have shown that if the cell's retention time is relaxed, the P/E cycle is improved significantly [37], [38]. More specifically, [38] states

that if the cell retention is reduced from 1 year to 3 days, the P/E cycle endurance is increased by $50\times$. With retention time relaxation, we take a similar approach as in [26] to estimate the lifetime of the OptimStore device. 3D pSLC is known to support ten times larger P/E cycles compared to TLC, reaching up to $30K\sim 40K$ cycles [7]. The tensors stored in the reservoir area are short-lived and does not require a long retention time. Thus, we can safely relax the retention time to 3 days. OptimStore device with 1TB of pSLC NAND flash memory reservoir can potentially support up to 1,500,000 TBW ($= 30K \text{ P/E cycles} \times 50 \times 1TB$). This translates into 12.4 million iterations of training of the 10.3B GPT model, as it consumes 124GBW for every iteration to update optimizer states. Considering the iteration time, OptimStore can support up to 9.4 years of training, which is longer than a typical 5-year warranty for commercial SSDs.

Bypassing ECC On-die ECC logic can enhance reliability. However, we argue that ECC logic is non-essential in OptimStore for several reasons. First, while the dominant error source of 3D NAND flash devices is retention loss [39], [45], tensors are stored in the OptimStore device only for a brief moment, hence highly unlikely to be affected by retention error. Second, OptimStore is implemented with pSLC flash devices, which are significantly less prone to bit errors. Single-level cells maintain bit error rate (BER) lower than 10^{-8} even with hundreds of thousands of P/E cycles [4]. OptimStore employs simple comparators to flip back critical bit error in MSB of exponent if the erroneous input value is NaN. Other than the NaN filters, complex ECC logic is unnecessary as the BER level around 10^{-8} showed negligible effect on the accuracy, thanks to the inherent error-resiliency of DNN.

IX. RELATED WORKS

In-Storage Processing Concept of bringing computation to storage is not new. In particular, many works have proposed ISP as an attempt to accelerate data center applications such as query processing [10], [28], [29], [36], [40], [55], and key-value store [55], check pointing [1], [65] and data analytics [60], [61]. Genome sequencing [41], pattern matching [18], [20] was also explored as targets of ISP acceleration. Although prior works share the underlying philosophy of near-data processing, the workload characteristic is different from OptimStore. They either handle I/O requests with irregular access patterns or accelerate read-intensive workloads. OptimStore is designed for both read and write-intensive DNN optimizer step. It utilizes flash memory as if it was a DRAM main memory. Additionally, these works, except for [40], processes data in the embedded core or with extra accelerator added to the controller. OptimStore goes deeper inside the storage device and performs computations on the flash dies.

Accelerating DNN training with ISP In recent years, as deep learning models in various fields grow in size, many works have explored the possibility of a flash-based storage device as a solution for accelerating large DNN training. Behemoth [26] proposes a flash-centric architecture for training large-scale transformer models where all the tensors are stored in the

flash memory. Behemoth and OptimStore share the same goal: enabling large-scale DNN training. However, while OptimStore targets the weight update stage under NVMe offloading scheme in the conventional computing platform; Behemoth is designed as a standalone accelerator to replace GPU. Apart from NLP models, ISP solutions like RecSSD [64] and [25] target embedding layer of recommendation systems while [31], [33] suggests ISP acceleration of graph neural networks. Some works [35] focus on more general DNN workloads with CNN. None of these works point out that the bus contention in the flash channel hinders the performance of the ISP and processes data in a separate logic unit placed near the storage. **On-die Processing** So far, only a few works have suggested placing logic inside the flash die. 'Chip level' accelerator in Deepstore [40] has a systolic array on the flash die to perform a deep learning-based query search. It is not feasible to place complex logic component inside the die, and Deepstore lacks details of how the logic concerts with the flash interface. [14] are similar to OptimStore in the sense that its workload is bulk elementwise vector operations. However, it lacks consideration of existing protocol and system support, including FTL. Moreover, its application is limited to hyper-dimensional computing [12] and does not support computations that require data from multiple planes.

GradPIM [23] is an NDP solution based on DRAM with a DDR4 interface targeting the weight update stage of DNN training. GradPIM and OptimStore are implemented in totally different memory systems, and due to the limitation of DRAM capacity, GradPIM is not suited for large-scale DNN training.

X. CONCLUSION

We propose OptimStore, an on-die processing architecture for the large-scale DNN optimization stage. By adding minimal logic units while leveraging internal registers of NAND flash dies, we enable on-die computations capable of performing high-precision math operations required for advanced DNN optimizers. Our simple but feasible design can be easily integrated into the ONFI protocol as standard flash commands. OptimStore benefits from exploiting SSD's maximal structural and flash-level parallelism by processing data inside the die and avoiding channel contentions. OptimStore also has superior scalability, free from data traffic over flash channels. OptimStore brought $2.4\times$ speedup in end-to-end training time and reduced energy consumption to $3.6\times$ compared to the baseline SSD offloading.

ACKNOWLEDGEMENTS

This work was supported in part by BK21 FOUR and MSIT (Ministry of Science and ICT), Korea, under the ITRC (Information Technology Research Center) support program under Grant IITP-2020-0-01847, in part by the IITP (Institute of Information Communications Technology Planning Evaluation). This work was also supported Future Non-volatile Memory Cluster Academia Collaboration Program funded by Samsung Electronics. The EDA tool was supported by the IC Design Education Center (IDEC), Korea.

REFERENCES

- [1] A. Agrawal, G. H. Loh, and J. Tuck, "Leveraging near data processing for high-performance checkpoint/restart," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2017, pp. 1–12.
- [2] J. Bae, J. Lee, Y. Jin, S. Son, S. Kim, H. Jang, T. J. Ham, and J. W. Lee, "Flashneuron: Ssd-enabled large-batch training of very deep neural networks," in *19th USENIX Conference on File and Storage Technologies (FAST 21)*, 2021, pp. 387–401.
- [3] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," *arXiv preprint arXiv:2005.14165*, 2020.
- [4] Y. Cai, E. F. Haratsch, O. Mutlu, and K. Mai, "Error patterns in mlc nand flash memory: Measurement, characterization, and analysis," in *2012 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2012, pp. 521–526.
- [5] H. Choe, S. Lee, H. Nam, S. Park, S. Kim, E.-Y. Chung, and S. Yoon, "Near-data processing for differentiable machine learning models," *arXiv preprint arXiv:1610.02273*, 2016.
- [6] C. Choi, "Multi-stream write ssd," *Flash Memory Summit*, 2016. [Online]. Available: <https://www.overleaf.com/project/624d8b21a7ab0b664da80282>
- [7] . K. Corporation, "Kioxia technical brief." [Online]. Available: https://business.kioxia.com/content/dam/kioxia/ncsa/en-us/business/memory/asset/KIOXIA_pSLC_for_IoT_devices_Tech_Brief.pdf
- [8] D. Das, N. Mellempudi, D. Mudigere, D. Kalamkar, S. Avancha, K. Banerjee, S. Sridharan, K. Vaidyanathan, B. Kaul, E. Georganas, A. Heinecke, P. Dubey, N. Shustrov, R. Dubtsov, E. Fomenko, and V. Pirogov, "Mixed precision training of convolutional neural networks using integer operations," *arXiv preprint arXiv:1802.00930*, 2018.
- [9] Y. N. Dauphin, H. de Vries, J. Chung, and Y. Bengio, "Rmsprop and equilibrated adaptive learning rates for non-convex optimization." *CoRR*, vol. abs/1502.04390, 2015.
- [10] J. Do, Y.-S. Kee, J. M. Patel, C. Park, K. Park, and D. J. DeWitt, "Query processing on smart ssds: Opportunities and challenges," in *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*, 2013, pp. 1221–1230.
- [11] J. Duchi, E. Hazan, and Y. Singer, "Adaptive subgradient methods for online learning and stochastic optimization," *Journal of Machine Learning Research*, vol. 12, no. Jul, pp. 2121–2159, 2011.
- [12] L. Ge and K. K. Parhi, "Classification using hyperdimensional computing: A review," *IEEE Circuits and Systems Magazine*, vol. 20, no. 2, pp. 30–47, 2020.
- [13] D. Gouk, M. Kwon, J. Zhang, S. Koh, W. Choi, N. S. Kim, M. Kandemir, and M. Jung, "Amber*: Enabling precise full-system simulation with detailed modeling of all ssd resources," in *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2018, pp. 469–481.
- [14] S. Gupta, J. Morris, M. Imani, R. Ramkumar, J. Yu, A. Tiwari, B. Aksanli, and T. Š. Rosing, "Thrifty: Training with hyperdimensional computing across flash hierarchy," in *2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 2020, pp. 1–9.
- [15] M. Hildebrand, J. Khan, S. Trika, J. Lowe-Power, and V. Akella, "Autotm: Automatic tensor movement in heterogeneous memory systems using integer linear programming," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2020, pp. 875–890.
- [16] C.-C. Huang, G. Jin, and J. Li, "Swapadvisor: Pushing deep learning beyond the gpu memory limit via smart swapping," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2020, pp. 1341–1355.
- [17] O. N. F. Interface, "Onfi 5.0 specification." [Online]. Available: <https://www.onfi.org/specifications>
- [18] W. S. Jeong, C. Lee, K. Kim, M. K. Yoon, W. Jeon, M. Jung, and W. W. Ro, "React: Scalable and high-performance regular expression pattern matching accelerator for in-storage processing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 5, pp. 1137–1151, 2019.
- [19] X. Jia, S. Song, W. He, Y. Wang, H. Rong, F. Zhou, L. Xie, Z. Guo, Y. Yang, L. Yu, T. Chen, G. Hu, S. Shi, and X. Chu, "Highly scalable deep learning training system with mixed-precision: Training imagenet in four minutes," *arXiv preprint arXiv:1807.11205*, 2018.
- [20] S.-W. Jun, H. T. Nguyen, V. Gadepally, and Arvind, "In-storage embedded accelerator for sparse pattern processing," in *2016 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 2016, pp. 1–7.
- [21] D. Kang, M. Kim, S. C. Jeon, W. Jung, J. Park, G. Choo, D.-k. Shim, A. Kavala, S.-B. Kim, K.-M. Kang, J. Lee, K. Ko, H.-W. Park, B.-J. Min, C. Yu, S. Yun, N. Kim, Y. Jung, S. Seo, S. Kim, M. K. Lee, J.-Y. Park, J. C. Kim, Y. S. Cha, K. Kim, Y. Jo, H. Kim, Y. Choi, J. Byun, J.-h. Park, K. Kim, T.-H. Kwon, Y. Min, C. Yoon, Y. Kim, D.-H. Kwak, E. Lee, W.-g. Hahn, K.-s. Kim, K. Kim, E. Yoon, W.-T. Kim, I. Lee, S. h. Moon, J. Ihm, D. S. Byeon, K.-W. Song, S. Hwang, and K. H. Kyung, "13.4 a 512gb 3-bit/cell 3d 6 th-generation v-nand flash memory with 82mb/s write throughput and 1.2 gb/s interface," in *2019 IEEE International Solid-State Circuits Conference (ISSCC)*. IEEE, 2019, pp. 216–218.
- [22] Karthik Chandrasekar, Christian Weis, Yonghui Li, Sven Goossens, Matthias Jung, Omar Naji, Benny Akesson, Norbert Wehn, and Kees Goossens, "DRAMPower: Open-source DRAM Power Energy Estimation Tool." [Online]. Available: <http://www.drampower.info>
- [23] H. Kim, H. Park, T. Kim, K. Cho, E. Lee, S. Ryu, H.-J. Lee, K. Choi, and J. Lee, "Gradpim: A practical processing-in-dram architecture for gradient descent," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 249–262.
- [24] M. Kim and S. Lee, "Reducing tail latency of dnn-based recommender systems using in-storage processing," in *Proceedings of the 11th ACM SIGOPS Asia-Pacific Workshop on Systems*, 2020, pp. 90–97.
- [25] M. Kim and S. Lee, "Reducing tail latency of dnn-based recommender systems using in-storage processing," in *Proceedings of the 11th ACM SIGOPS Asia-Pacific Workshop on Systems*, 2020, pp. 90–97.
- [26] S. Kim, Y. Jin, G. Sohn, J. Bae, T. J. Ham, and J. W. Lee, "Behemoth: A flash-centric training accelerator for extreme-scale dnns," in *19th USENIX Conference on File and Storage Technologies (FAST 21)*, 2021, pp. 371–385.
- [27] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.
- [28] G. Koo, K. K. Matam, I. Te, H. K. G. Narra, J. Li, H.-W. Tseng, S. Swanson, and M. Annaram, "Summarizer: trading communication with computing near storage," in *2017 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2017, pp. 219–231.
- [29] G. Koo, K. K. Matam, I. Te, H. K. G. Narra, J. Li, H.-W. Tseng, S. Swanson, and M. Annaram, "Summarizer: trading communication with computing near storage," in *2017 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2017, pp. 219–231.
- [30] O. Kuchaiev, B. Ginsburg, I. Gitman, V. Lavrukhin, J. Li, H. Nguyen, C. Case, and P. Micikevicius, "Mixed-precision training for nlp and speech recognition with openseq2seq," *arXiv preprint arXiv:1805.10387*, 2018.
- [31] M. Kwon, D. Gouk, S. Lee, and M. Jung, "Hardware/software co-programmable framework for computational ssds to accelerate deep learning service on large-scale graphs," *arXiv preprint arXiv:2201.09189*, 2022.
- [32] Y. Kwon and M. Rhu, "Beyond the memory wall: A case for memory-centric hpc system for deep learning," in *Proceedings of the 51st Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO-51. IEEE Press, 2018, p. 148–161.
- [33] C. Li, Y. Wang, C. Liu, S. Liang, H. Li, and X. Li, "Glist: Towards in-storage graph learning," in *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, 2021, pp. 225–238.
- [34] S. Li, J. H. Ahn, R. D. Strong, J. B. Brockman, D. M. Tullsen, and N. P. Jouppi, "Mcpat: An integrated power, area, and timing modeling framework for multicore and manycore architectures," in *Proceedings of the 42nd annual ieee/acm international symposium on microarchitecture*, 2009, pp. 469–480.
- [35] S. Liang, Y. Wang, C. Liu, H. Li, and X. Li, "Ins-dla: An in-ssd deep learning accelerator for near-data processing," in *2019 29th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, 2019, pp. 173–179.

- [36] S. Liang, Y. Wang, Y. Lu, Z. Yang, H. Li, and X. Li, "Cognitive {SSD}: A deep learning engine for {In-Storage} data retrieval," in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, 2019, pp. 395–410.
- [37] R.-S. Liu, C.-L. Yang, and W. Wu, "Optimizing nand flash-based ssds via retention relaxation," *Target*, vol. 11, no. 10, p. 00, 2012.
- [38] Y. Luo, Y. Cai, S. Ghose, J. Choi, and O. Mutlu, "Warm: Improving nand flash memory lifetime with write-hotness aware retention management," in *2015 31st Symposium on Mass Storage Systems and Technologies (MSST)*. IEEE, 2015, pp. 1–14.
- [39] Y. Luo, S. Ghose, Y. Cai, E. F. Haratsch, and O. Mutlu, "Improving 3d nand flash memory lifetime by tolerating early retention loss and process variation," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 2, no. 3, pp. 1–48, 2018.
- [40] V. S. Mailthody, Z. Qureshi, W. Liang, Z. Feng, S. G. De Gonzalo, Y. Li, H. Franke, J. Xiong, J. Huang, and W.-m. Hwu, "Deepstore: In-storage acceleration for intelligent queries," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 224–238.
- [41] N. Mansouri Ghiasi, J. Park, H. Mustafa, J. Kim, A. Olgun, A. Gollwitzer, D. Senol Cali, C. Firtina, H. Mao, N. Almadhoun Alser, R. Ausavarungnirun, N. Vijaykumar, M. Alser, and O. Mutlu, "Genstore: a high-performance in-storage processing system for genome sequence analysis," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022, pp. 635–654.
- [42] O. Michael, "Nand flash memory having internal ecc processing and method of operation thereof," U.S. Patent 9 367 392, Jun. 2016. [Online]. Available: <https://patents.google.com/patent/US20160034346A1/en>
- [43] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, "Mixed precision training," in *International Conference on Learning Representations*, 2018.
- [44] Microsoft, "DeepSpeed." [Online]. Available: <https://github.com/microsoft/DeepSpeed>
- [45] J. Park, M. Kim, M. Chun, L. Orosa, J. Kim, and O. Mutlu, "Reducing solid-state drive read latency by optimizing read-retry," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2021, pp. 702–716.
- [46] S. Pati, S. Aga, N. Jayasena, and M. D. Sinclair, "Demystifying bert: Implications for accelerator design," *arXiv preprint arXiv:2104.08335*, 2021.
- [47] X. Peng, X. Shi, H. Dai, H. Jin, W. Ma, Q. Xiong, F. Yang, and X. Qian, "Capuchin: Tensor-based gpu memory management for deep learning," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2020, pp. 891–905.
- [48] M. E. Peters, M. Neumann, M. Iyyer, M. Gardner, C. Clark, K. Lee, and L. Zettlemoyer, "Deep contextualized word representations," 2018.
- [49] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.
- [50] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *arXiv preprint arXiv:1910.10683*, 2019.
- [51] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "Zero: Memory optimizations toward training trillion parameter models," in *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2020, pp. 1–16.
- [52] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, "Zero-infinity: Breaking the gpu memory wall for extreme scale deep learning," *arXiv preprint arXiv:2104.07857*, 2021.
- [53] J. Ren, J. Luo, K. Wu, M. Zhang, H. Jeon, and D. Li, "Sentinel: Efficient tensor migration and allocation on heterogeneous memory systems for deep learning," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 598–611.
- [54] Seagate, "Firecuda530 pcie 4.0 ssd." [Online]. Available: <https://www.seagate.com/kr/ko/products/gaming-drives/pc-gaming/firecuda-530-ssd/>
- [55] S. Seshadri, M. Gahagan, S. Bhaskaran, T. Bunker, A. De, Y. Jin, Y. Liu, and S. Swanson, "Willow: A user-programmable ssd," in *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*, 2014, pp. 67–80.
- [56] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2014.
- [57] R. Stoica, R. Pletka, N. Ioannou, N. Papandreou, S. Tomic, and H. Pozidis, "Understanding the design trade-offs of hybrid flash controllers," in *2019 IEEE 27th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*. IEEE, 2019, pp. 152–164.
- [58] I. Synopsys, "Design compiler." [Online]. Available: <https://www.synopsys.com/implementation-and-signoff/rtl-synthesis-test/design-compiler-graphical.html>
- [59] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 1–9.
- [60] D. Tiwari, S. Boboila, S. Vazhkudai, Y. Kim, X. Ma, P. Desnoyers, and Y. Solihin, "Active flash: Towards {Energy-Efficient},{In-Situ} data analytics on {Extreme-Scale} machines," in *11th USENIX Conference on File and Storage Technologies (FAST 13)*, 2013, pp. 119–132.
- [61] D. Tiwari, S. S. Vazhkudai, Y. Kim, X. Ma, S. Boboila, and P. J. Desnoyers, "Reducing data movement costs using {Energy-Efficient}, active computation on {SSD}," in *2012 Workshop on Power-Aware Computing and Systems (HotPower 12)*, 2012.
- [62] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017.
- [63] C. J. Walczyk, L. V. Moroz, and J. L. Cieřliński, "Improving the accuracy of the fast inverse square root algorithm," 2018.
- [64] M. Wilkening, U. Gupta, S. Hsia, C. Trippel, C.-J. Wu, D. Brooks, and G.-Y. Wei, "Recssd: near data processing for solid state drive based recommendation inference," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2021, pp. 717–729.
- [65] J. Yoon, W. S. Jeong, and W. W. Ro, "Check-in: In-storage check-pointing for key-value store system leveraging flash-based ssds," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 693–706.